

新哲文院算法社 2026-2027 学年 C++ 教材 · 答案与解析

新哲文院算法社 2026-2027 学年 C++ 教材 · 答案与解析

本文件对应 `C++教材_新哲文院算法社_2026-2027.md`

按教材顺序排列，包含课堂练习和课后作业的完整解答与解析

第一部分：基础掌握 · 答案

第一单元 · 课堂练习 1.1

题目：编写一个程序，输入两个整数 a 和 b，输出它们的和、差、积、商（整除）。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int a, b;
6     cin >> a >> b;
7
8     cout << "和=" << a + b
9         << " 差=" << a - b
10        << " 积=" << a * b
11        << " 商=" << a / b << endl;
12
13     return 0;
14 }
```

✎ 解析

- `cin >> a >> b` 可以连续读取多个变量，用空格分隔输入
 - `a / b` 是整数除法，自动截断小数部分（如 $10/3=3$ ）
 - 如果 `b` 可能为 0，需要加判断防止运行时错误
 - 输出用 `endl` 换行，或者用 `'\n'`（更快）
-

第二单元 · 课堂练习 2.1

练习1：输入圆的半径，输出面积和周长（保留两位小数）。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     const double PI = 3.1415926535;
9     double r;
10    cin >> r;
11
12    double area = PI * r * r;
13    double circumference = 2 * PI * r;
14
15    printf("面积=%.2f 周长=%.2f\n", area, circumference);
16
17    return 0;
18 }
```

解析

- 用 `double` 存储半径和结果，精度更高
- `printf` 的 `%.2f` 表示保留 2 位小数
- 也可以用 `cout << fixed << setprecision(2) << area`
- `const double PI` 定义为常量，避免硬编码

练习2：输入一个三位数，输出它的百位、十位、个位。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     int hundreds = n / 100;           // 百位：整除100
9     int tens = (n / 10) % 10;         // 十位：先整除10再取余10
10    int ones = n % 10;                 // 个位：取余10
11
12    cout << "百位=" << hundreds
13         << " 十位=" << tens
14         << " 个位=" << ones << endl;
15
16    return 0;
17 }
```

📝 解析

- $n / 100$ 整除得到百位（如 $365/100=3$ ）
- $(n / 10) \% 10$ 先去掉个位再取新数的个位（ $365/10=36, 36\%10=6$ ）
- $n \% 10$ 取余得到个位
- ⚠ 注意顺序：先除后取余，不要搞反

练习3：输入两个整数 a 和 b，输出 a 除以 b 的商和余数。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int a, b;
6     cin >> a >> b;
7
8     cout << "商=" << a / b << " 余数=" << a % b << endl;
9
10    return 0;
11 }
```

解析

- `/` 是整除运算符，得到商
- `%` 是取余运算符，得到余数
- 前提： $b \neq 0$ ，否则会运行时错误

第三单元 • 课堂练习 3.1

练习 1：判断闰年。输入年份，输出是否为闰年。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int year;
6     cin >> year;
7
8     bool isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 ==
    0);
9
10    if (isLeap) {
11        cout << year << "是闰年" << endl;
12    } else {
13        cout << year << "不是闰年" << endl;
14    }
15
16    return 0;
17 }
```

解析

- 闰年规则：能被 4 整除但不能被 100 整除，**或**能被 400 整除
- 用括号明确优先级，避免逻辑错误
- 也可以写成函数 `bool isLeapYear(int y)`

练习 2：输出 1~100 中所有能被 3 整除但不能被 5 整除的数。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     for (int i = 1; i <= 100; i++) {
6         if (i % 3 == 0 && i % 5 != 0) {
7             cout << i << " ";
8         }
9     }
10    cout << endl;
11    return 0;
12 }
```

解析

- `i % 3 == 0` 表示能被 3 整除
- `i % 5 != 0` 表示不能被 5 整除
- `&&` 表示两个条件同时满足

练习3：打印三角形（输入行数 n）。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= i; j++) {
10             cout << "*";
11         }
12         cout << endl;
13     }
14
15     return 0;
16 }
```

解析

- 外层循环控制行数
- 内层循环控制每行的星号数量（第 i 行有 i 个星号）
- 这是嵌套循环的基础练习

第四单元 • 课堂练习 4.1

练习 1：输入 n 个整数，用 vector 存储，输出去重后的元素个数和去重后的序列（保持原顺序）。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n;
9     cin >> n;
10
11     vector<int> v;
12     unordered_set<int> seen; // 用于去重
13
14     for (int i = 0; i < n; i++) {
15         int x;
16         cin >> x;
17         if (!seen.count(x)) { // 如果没见过这个数
18             seen.insert(x);
19             v.push_back(x);
20         }
21     }
22
23     cout << v.size() << endl;
24     for (int x : v) {
25         cout << x << " ";
26     }
27     cout << endl;
28
29     return 0;
30 }
```

解析

- `unordered_set` 的查找是 $O(1)$ ，比遍历 `vector` 找快得多
- 用 `seen` 记录已出现的数，保证原顺序
- 也可以用 `set` 自动排序，但会丢失原顺序

练习2：输入一个字符串，统计每个字符出现的次数，按字符 ASCII 顺序输出。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     map<char, int> cnt; // 自动按 key 排序
9
10    for (char c : s) {
11        cnt[c]++;
12    }
13
14    for (auto& kv : cnt) {
15        cout << kv.first << ":" << kv.second << endl;
16    }
17
18    return 0;
19 }
```

解析

- `map<char, int>` 自动按字符 ASCII 码排序
- `cnt[c]++` 如果 `c` 不存在会自动创建并初始化为 0
- `auto& kv` 中 `kv.first` 是字符, `kv.second` 是次数

第五单元 • 课堂练习 5.1

练习 1: 写一个函数 `bool isPrime(int n)` 判断 `n` 是否为素数。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPrime(int n) {
5     if (n < 2) return false;           // 小于2不是素数
6     if (n == 2) return true;           // 2是素数
7     if (n % 2 == 0) return false;      // 偶数不是素数
8
9     for (int i = 3; i * i <= n; i += 2) { // 只需检查到√n
10         if (n % i == 0) return false;
11     }
12     return true;
13 }
14
15 int main() {
16     int n;
17     cin >> n;
18     cout << (isPrime(n) ? "true" : "false") << endl;
19     return 0;
20 }
```

解析

- 小于2的数不是素数
- 只需检查到 $\sqrt{n}$ (即 $i*i \leq n$)，超过这个范围如果有因子一定已经找到了
- 跳过偶数 ($i += 2$)，效率翻倍
- 时间复杂度 $O(\sqrt{n})$

练习2：写一个函数 `int gcd(int a, int b)` 用辗转相除法求最大公约数。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int gcd(int a, int b) {
5     while (b != 0) {
6         int temp = b;
7         b = a % b;
8         a = temp;
9     }
10    return a;
11 }
12
13 int main() {
14     int a, b;
15     cin >> a >> b;
16     cout << gcd(a, b) << endl;
17     return 0;
18 }
```

解析

- 辗转相除法: `gcd(a,b) = gcd(b, a%b)` , 直到 `b=0`
- 也可以用递归: `if (b==0) return a; return gcd(b, a%b);`
- C++17 起有内置 `std::gcd` , 但竞赛中建议自己写

第六单元 • 课堂练习 6.1

题目: 从 `data.txt` 读取若干整数, 计算和与平均值, 写入 `result.txt` 。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ifstream infile("data.txt");
6     ofstream outfile("result.txt");
7
8     int x;
9     int sum = 0;
10    int count = 0;
11
12    while (infile >> x) {
13        sum += x;
14        count++;
15    }
16
17    double avg = (double)sum / count;
18
19    outfile << "总和=" << sum << endl;
20    outfile << fixed << setprecision(2) << "平均值=" << avg << endl;
21
22    infile.close();
23    outfile.close();
24
25    return 0;
26 }
```

解析

- `ifstream` 用于读文件, `ofstream` 用于写文件
- `while (infile >> x)` 持续读取直到文件结束
- `(double)sum / count` 强制转换避免整数除法
- 记得关闭文件 (离开作用域会自动关闭)

基础模块 • 课后作业答案

作业1: 判断闰年。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int year;
6     cin >> year;
7
8     if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
9         cout << "Leap Year" << endl;
10    } else {
11        cout << "Not Leap Year" << endl;
12    }
13
14    return 0;
15 }
```

解析

- 闰年必须满足“能被4整除且不能被100整除”，或者“能被400整除”
- 输出字符串必须与题目要求的 `Leap Year` 和 `Not Leap Year` 完全一致

作业2：求 $1+2+3+\dots+n$ 的和。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     long long sum = 0; // 用 long long 防止溢出
9     for (int i = 1; i <= n; i++) {
10         sum += i;
11     }
12
13     cout << sum << endl;
14     return 0;
15 }
```

解析

- 也可以用公式 $n*(n+1)/2$ 一步算出
- n 较大时 `sum` 可能超过 `int` 范围，用 `long long`

解析

- 循环从 1 累加到 n ，正好实现题目要求的循环方法
- 使用 `long long` 保存结果，避免较大的 n 导致 `int` 溢出

作业3：判断回文串（忽略大小写）。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPalindrome(string s) {
5     // 先转小写
6     for (char& c : s) {
7         c = tolower(c);
8     }
9
10    int left = 0, right = s.length() - 1;
11    while (left < right) {
12        if (s[left] != s[right]) return false;
13        left++;
14        right--;
15    }
16    return true;
17 }
18
19 int main() {
20     string s;
21     cin >> s;
22     cout << (isPalindrome(s) ? "是回文" : "不是回文") << endl;
23     return 0;
24 }
```

解析

- 使用 `tolower` 将字符统一转换为小写，再用双指针比较字符串首尾
- `left` 和 `right` 向中间移动，发现任意一对字符不同即可判定不是回文
- 这里用 `cin >> s`，输入字符串不能包含空格；若要支持空格应改用 `getline`

作业4：统计成绩。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int scores[10];
6     int mx = -1, mn = 101, sum = 0;
7
8     for (int i = 0; i < 10; i++) {
9         cin >> scores[i];
10        sum += scores[i];
11        if (scores[i] > mx) mx = scores[i];
12        if (scores[i] < mn) mn = scores[i];
13    }
14
15    double avg = (double)sum / 10;
16
17    cout << "最高分=" << mx << endl;
18    cout << "最低分=" << mn << endl;
19    cout << fixed << setprecision(2) << "平均分=" << avg << endl;
20
21    int above = 0;
22    for (int i = 0; i < 10; i++) {
23        if (scores[i] > avg) above++;
24    }
25    cout << "高于平均分的人数=" << above << endl;
26
27    return 0;
28 }
```

解析

- 数组固定读取 10 个成绩，并在读取过程中同步计算总和、最高分和最低分
- 平均分使用浮点除法；最后再次遍历数组，统计严格高于平均分的人数

作业 5：打印菱形。

解答


```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void printDiamond(int n) {
5     if (n <= 0) return;
6
7     // 按题目要求总共输出 n 行；奇数 n 时为标准对称菱形
8     int half = (n + 1) / 2;
9     int maxWidth = 2 * half - 1;
10    for (int row = 1; row <= n; row++) {
11        int level = row <= half ? row : n - row + 1;
12        int width = 2 * level - 1;
13        for (int j = 0; j < (maxWidth - width) / 2; j++) cout << " ";
14        for (int j = 0; j < width; j++) cout << "*";
15        cout << endl;
16    }
17 }
18
19 int main() {
20     int n;
21     cin >> n;
22     printDiamond(n);
23     return 0;
24 }
```

解析

- `level` 先递增后递减，因此总输出行数严格为 `n`
- 第 `level` 层星号数为 `2*level - 1`
- 空格数根据最大宽度计算，使每一行保持居中

第二部分：竞赛进阶 · 答案

第五单元 · 课堂练习 5.1

练习1：用冒泡排序对字符串数组按字典序从小到大排序。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     vector<string> words;
9     string s;
10    while (cin >> s) {
11        words.push_back(s);
12    }
13
14    // 冒泡排序（按字典序）
15    for (int i = 0; i < words.size() - 1; i++) {
16        for (int j = 0; j < words.size() - 1 - i; j++) {
17            if (words[j] > words[j + 1]) { // string 支持直接用 > 比较
18                swap(words[j], words[j + 1]);
19            }
20        }
21    }
22
23    for (string& w : words) {
24        cout << w << " ";
25    }
26    cout << endl;
27
28    return 0;
29 }
```

📝 解析

- C++ 的 `string` 类型重载了比较运算符，按字典序比较
- `"apple" < "banana" → true`（因为 'a' < 'b'）
- 输入用 `while (cin >> s)` 读取直到 EOF

练习2：用桶排序统计 0~9 每个数字出现次数。

✓ 解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     int buckets[10] = {0}; // 10个桶，初始为0
9
10    for (int i = 0; i < n; i++) {
11        int x;
12        cin >> x;
13        buckets[x]++;
14    }
15
16    for (int i = 0; i <= 9; i++) {
17        cout << i << ":" << buckets[i] << endl;
18    }
19
20    return 0;
21 }
```

解析

- 桶的下标就是数字本身，值为出现次数
- 这是计数排序的最简形式，时间复杂度 $O(n)$
- 输出时遍历桶 0~9 即可

第五单元 · 课后作业 5

作业 1：实现冒泡排序的降序版本。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int arr[105];
8
9     for (int i = 0; i < n; i++) cin >> arr[i];
10
11     // 冒泡降序：把小的往后"冒"
12     for (int i = 0; i < n - 1; i++) {
13         for (int j = 0; j < n - 1 - i; j++) {
14             if (arr[j] < arr[j + 1]) { // 注意：小于号（升序是大于号）
15                 swap(arr[j], arr[j + 1]);
16             }
17         }
18     }
19
20     for (int i = 0; i < n; i++) {
21         cout << arr[i] << " ";
22     }
23     cout << endl;
24
25     return 0;
26 }
```

解析

- 降序只需把比较符号从 `>` 改为 `<`
- 每一轮把当前最小的"冒泡"到最后

作业2：用桶排序按成绩从高到低输出学生姓名。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n;
9     cin >> n;
10
11     // 每个成绩对应一个名字列表 (vector)，保持输入顺序
12     vector<string> buckets[101];
13
14     for (int i = 0; i < n; i++) {
15         string name;
16         int score;
17         cin >> name >> score;
18         buckets[score].push_back(name);
19     }
20
21     // 从100到0遍历（从高到低）
22     for (int i = 100; i >= 0; i--) {
23         for (string& name : buckets[i]) {
24             cout << name << " ";
25         }
26     }
27     cout << endl;
28
29     return 0;
30 }
```

解析

- 每个桶是一个 `vector<string>`，存储该成绩的所有学生
- 从 100 遍历到 0 实现从高到低输出
- 同一成绩的学生按输入顺序输出（vector 保持插入顺序）

练习1：输出 1~n 中所有和为 n 的组合（每个数只能用一次）。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n;
5 vector<int> path;
6
7 void dfs(int start, int remain) {
8     if (remain == 0) {
9         for (int i = 0; i < path.size(); i++) {
10             if (i > 0) cout << " ";
11             cout << path[i];
12         }
13         cout << endl;
14         return;
15     }
16
17     for (int i = start; i <= remain; i++) {
18         path.push_back(i);
19         dfs(i + 1, remain - i); // i+1 保证不重复使用
20         path.pop_back();
21     }
22 }
23
24 int main() {
25     cin >> n;
26     dfs(1, n);
27     return 0;
28 }
```

✍ 解析

- `start` 参数保证每个数只用一次且组合不重复
- `remain` 记录还需要凑多少
- 当 `remain == 0` 时说明找到了一个合法组合

练习2：N 皇后问题，输出一种可行方案。

✓ 解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n;
5 vector<int> cols; // cols[i] = 第i行皇后所在的列
6 bool found = false;
7
8 bool isValid(int row, int col) {
9     for (int i = 0; i < row; i++) {
10         if (cols[i] == col) return false; // 同列
11         if (abs(cols[i] - col) == abs(i - row)) return false; // 同对角线
12     }
13     return true;
14 }
15
16 void dfs(int row) {
17     if (found) return; // 只找一种方案
18
19     if (row == n) {
20         for (int i = 0; i < n; i++) {
21             for (int j = 0; j < n; j++) {
22                 if (cols[i] == j) cout << "Q ";
23                 else cout << ". ";
24             }
25             cout << endl;
26         }
27         found = true;
28         return;
29     }
30
31     for (int col = 0; col < n; col++) {
32         if (isValid(row, col)) {
33             cols[row] = col;
34             dfs(row + 1);
35         }
36     }
37 }
38
39 int main() {
40     cin >> n;
41     cols.resize(n);
42     dfs(0);
```



```
43     return 0;
44 }
```

解析

- 按行放置皇后，每行选一个列
 - `isValid` 检查：同列、主对角线、副对角线
 - 对角线判断技巧： `abs(col - cols[i]) == abs(row - i)`
-

第六单元 · 课后作业 6

作业 1：岛屿数量。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m;
5 vector<vector<int>> grid;
6 vector<vector<bool>> visited;
7
8 void dfs(int x, int y) {
9     visited[x][y] = true;
10
11     int dx[] = {-1, 0, 1, 0};
12     int dy[] = {0, 1, 0, -1};
13
14     for (int dir = 0; dir < 4; dir++) {
15         int nx = x + dx[dir];
16         int ny = y + dy[dir];
17         if (nx >= 0 && nx < n && ny >= 0 && ny < m
18             && grid[nx][ny] == 1 && !visited[nx][ny]) {
19             dfs(nx, ny);
20         }
21     }
22 }
23
24 int main() {
25     cin >> n >> m;
26     grid.resize(n, vector<int>(m));
27     visited.resize(n, vector<bool>(m, false));
28
29     for (int i = 0; i < n; i++) {
30         for (int j = 0; j < m; j++) {
31             cin >> grid[i][j];
32         }
33     }
34
35     int islands = 0;
36     for (int i = 0; i < n; i++) {
37         for (int j = 0; j < m; j++) {
38             if (grid[i][j] == 1 && !visited[i][j]) {
39                 dfs(i, j);
40                 islands++;
41             }
42         }
43     }
```

```
43     }  
44  
45     cout << islands << endl;  
46     return 0;  
47 }
```

解析

- 遍历每个格子，遇到未访问的陆地就 DFS 标记整个岛
- 每次发现新岛，计数器 +1
- DFS 中把连通的所有陆地都标记为已访问

作业 2：找出图中从起点到终点的所有路径。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, start, target;
5 vector<vector<int>> graph;
6 vector<bool> visited;
7 vector<int> path;
8
9 void dfs(int node) {
10     path.push_back(node);
11     visited[node] = true;
12
13     if (node == target) {
14         // 输出路径
15         for (int i = 0; i < path.size(); i++) {
16             if (i > 0) cout << " ";
17             cout << path[i];
18         }
19         cout << endl;
20     } else {
21         for (int neighbor : graph[node]) {
22             if (!visited[neighbor]) {
23                 dfs(neighbor);
24             }
25         }
26     }
27
28     // 回溯
29     visited[node] = false;
30     path.pop_back();
31 }
32
33 int main() {
34     cin >> n >> start >> target;
35     graph.resize(n + 1);
36     visited.resize(n + 1, false);
37
38     int edges;
39     cin >> edges;
40     for (int i = 0; i < edges; i++) {
41         int u, v;
42         cin >> u >> v;
```

```
43     graph[u].push_back(v);
44     graph[v].push_back(u); // 无向图
45 }
46
47 dfs(start);
48 return 0;
49 }
```

解析

- `path` 保存当前从起点出发的路径，`visited` 防止在环中重复访问
 - 到达目标节点时输出当前路径；函数返回前撤销当前节点的访问标记，实现回溯
 - 这是无向图的所有简单路径搜索，若图中路径数量很多，输出规模可能很大
-

第七单元 · 课堂练习 7.1

练习1：骑士（马）从 (0,0) 到 (7,7) 的最少步数。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int bfs() {
5     int n = 8;
6     vector<vector<bool>> visited(n, vector<bool>(n, false));
7     queue<pair<int, int>> q;
8
9     q.push({0, 0});
10    visited[0][0] = true;
11    int steps = 0;
12
13    // 马的8个方向
14    int dx[] = {2, 1, -1, -2, -2, -1, 1, 2};
15    int dy[] = {1, 2, 2, 1, -1, -2, -2, -1};
16
17    while (!q.empty()) {
18        int size = q.size();
19        for (int i = 0; i < size; i++) {
20            auto [x, y] = q.front();
21            q.pop();
22
23            if (x == 7 && y == 7) return steps;
24
25            for (int dir = 0; dir < 8; dir++) {
26                int nx = x + dx[dir];
27                int ny = y + dy[dir];
28                if (nx >= 0 && nx < n && ny >= 0 && ny < n && !visited
[nx][ny]) {
29                    visited[nx][ny] = true;
30                    q.push({nx, ny});
31                }
32            }
33        }
34        steps++;
35    }
36    return -1;
37 }
38
39 int main() {
40     cout << bfs() << endl;
```

```
41     return 0;  
42 }
```

解析

- BFS 保证找到的是最短路径
- 马有 8 个可能的移动方向
- 按层计数 `steps`，每层代表一步

练习 2：单词接龙最短变换序列长度。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int wordLadder(string beginWord, string endWord, vector<string>& wordList) {
5     unordered_set<string> dict(wordList.begin(), wordList.end());
6     if (!dict.count(endWord)) return 0;
7
8     queue<pair<string, int>> q; // (word, steps)
9     q.push({beginWord, 1});
10    dict.erase(beginWord);
11
12    while (!q.empty()) {
13        auto [word, steps] = q.front();
14        q.pop();
15
16        for (int i = 0; i < word.length(); i++) {
17            string next = word;
18            for (char c = 'a'; c <= 'z'; c++) {
19                next[i] = c;
20                if (next == endWord) return steps + 1;
21                if (dict.count(next)) {
22                    dict.erase(next);
23                    q.push({next, steps + 1});
24                }
25            }
26        }
27    }
28    return 0;
29 }
```

解析

- BFS 队列中的第二个值表示当前序列包含的单词数量
- 每次只替换一个字符，并且只把字典中的新单词加入队列
- BFS 按层搜索，因此第一次到达目标单词时得到最短变换序列

作业 1： 网格最短路径（含障碍物）。

 **解答**

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n, m;
9     cin >> n >> m;
10
11     vector<vector<int>> grid(n, vector<int>(m));
12     for (int i = 0; i < n; i++) {
13         for (int j = 0; j < m; j++) {
14             cin >> grid[i][j];
15         }
16     }
17
18     if (grid[0][0] == 1 || grid[n-1][m-1] == 1) {
19         cout << -1 << endl;
20         return 0;
21     }
22
23     vector<vector<bool>> visited(n, vector<bool>(m, false));
24     queue<pair<int, int>> q;
25     q.push({0, 0});
26     visited[0][0] = true;
27     int steps = 0;
28
29     int dx[] = {-1, 0, 1, 0};
30     int dy[] = {0, 1, 0, -1};
31
32     while (!q.empty()) {
33         int size = q.size();
34         for (int i = 0; i < size; i++) {
35             auto [x, y] = q.front();
36             q.pop();
37
38             if (x == n - 1 && y == m - 1) {
39                 cout << steps << endl;
40                 return 0;
41             }
42         }
```

```

43         for (int dir = 0; dir < 4; dir++) {
44             int nx = x + dx[dir];
45             int ny = y + dy[dir];
46             if (nx >= 0 && nx < n && ny >= 0 && ny < m
47                 && grid[nx][ny] == 0 && !visited[nx][ny]) {
48                 visited[nx][ny] = true;
49                 q.push({nx, ny});
50             }
51         }
52     }
53     steps++;
54 }
55
56 cout << -1 << endl;
57 return 0;
58 }

```

解析

- BFS 按距离分层，起点距离设为 0，因此到达相邻格子的距离为 1
- `visited` 保证每个可通行格子只入队一次
- 起点或终点是障碍物时直接输出 -1，搜索结束仍未到达终点时也输出 -1

作业 2：水壶问题。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool canMeasure(int a, int b, int c) {
5     if (c > a + b) return false;
6     if (c == 0) return true;
7
8     // BFS 搜索所有可能的状态
9     queue<pair<int, int>> q;
10    set<pair<int, int>> visited;
11    q.push({0, 0});
12    visited.insert({0, 0});
13
14    while (!q.empty()) {
15        auto [x, y] = q.front();
16        q.pop();
17
18        if (x == c || y == c || x + y == c) return true;
19
20        // 6种操作
21        vector<pair<int, int>> next_states = {
22            {a, y},          // 装满A
23            {x, b},          // 装满B
24            {0, y},          // 倒空A
25            {x, 0},          // 倒空B
26            {min(a, x + y), max(0, x + y - a)}, // B倒入A
27            {max(0, x + y - b), min(b, x + y)}  // A倒入B
28        };
29
30        for (auto& ns : next_states) {
31            if (!visited.count(ns)) {
32                visited.insert(ns);
33                q.push(ns);
34            }
35        }
36    }
37    return false;
38 }
39
40 int main() {
41     int a, b, c;
42     cin >> a >> b >> c;
```

```
43     cout << (canMeasure(a, b, c) ? "可以" : "不可以") << endl;
44     return 0;
45 }
```

解析

- 一个状态用 `(x, y)` 表示两个水壶当前的水量
- 每个状态可通过装满、倒空以及相互倒水得到 6 种转移
- 使用 `set` 记录已经访问过的状态，BFS 会穷举所有可达状态并避免重复搜索

第八单元 · 课堂练习 8.1

练习1：递归求阶乘。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long factorial(int n) {
5     if (n <= 1) return 1;           // 终止条件
6     return n * factorial(n - 1);    // 递归调用
7 }
8
9 int main() {
10     int n;
11     cin >> n;
12     cout << factorial(n) << endl;  // 5! = 120
13     return 0;
14 }
```

解析

- 终止条件： `n <= 1` 时返回 1
- 递归关系： `n! = n × (n-1)!`
- 用 `long long` 防止大数溢出（20! 就超过 int 范围）

练习2：递归实现字符串反转。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string reverseStr(string s) {
5     if (s.length() <= 1) return s;    // 终止条件
6     return reverseStr(s.substr(1)) + s[0]; // 递归：反转后面的 + 第一个
7 }
8
9 int main() {
10     string s;
11     cin >> s;
12     cout << reverseStr(s) << endl;    // hello → olleh
13     return 0;
14 }
```

📝 解析

- 把第一个字符放到最后，递归反转剩余部分
- `s.substr(1)` 返回从位置 1 开始到末尾的子串
- 也可以用双指针法（迭代）更高效

练习3：青蛙跳台阶（递归）。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long jump(int n) {
5     if (n <= 2) return n;           // f(1)=1, f(2)=2
6     return jump(n - 1) + jump(n - 2); // 从n-1跳1步 或 从n-2跳2步
7 }
8
9 int main() {
10     int n;
11     cin >> n;
12     cout << jump(n) << endl;
13     return 0;
14 }
```

解析

- 这其实就是斐波那契数列的变体
- 朴素递归效率低 ($O(2^n)$)，竞赛中应用记忆化或递推

第八单元 · 课后作业 8

作业 1：递归求最大公约数。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int gcd(int a, int b) {
5     if (b == 0) return a;           // 终止条件
6     return gcd(b, a % b);          // 递归调用
7 }
8
9 int main() {
10     int a, b;
11     cin >> a >> b;
12     cout << gcd(a, b) << endl;
13     return 0;
14 }
```

解析

- 递归关系为 $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ ，当 $b == 0$ 时返回 a
- 每次递归都会让第二个参数变小，最终一定到达终止条件
- 题目中的输入应为非负整数；若需要支持负数，可先对参数取绝对值

作业 2：递归判断回文。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPalindrome(string s, int left, int right) {
5     if (left >= right) return true;    // 终止条件
6     if (s[left] != s[right]) return false;
7     return isPalindrome(s, left + 1, right - 1); // 递归: 缩小范围
8 }
9
10 int main() {
11     string s;
12     cin >> s;
13     cout << (isPalindrome(s, 0, s.length() - 1) ? "是回文" : "不是回文") <
        < endl;
14     return 0;
15 }
```

解析

- `left` 和 `right` 分别指向字符串两端，递归比较并向中间收缩
- 两个指针相遇或交叉时，说明所有对应字符都相同，可以判定为回文
- 当前实现使用 `cin >> s`，只处理不含空格的字符串

作业3：递归生成所有子集。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> nums;
5 vector<int> path;
6
7 void dfs(int idx) {
8     if (idx == nums.size()) {
9         // 输出当前子集
10        cout << "{";
11        for (int i = 0; i < path.size(); i++) {
12            if (i > 0) cout << ",";
13            cout << path[i];
14        }
15        cout << "}" << endl;
16        return;
17    }
18
19    // 不选 nums[idx]
20    dfs(idx + 1);
21
22    // 选 nums[idx]
23    path.push_back(nums[idx]);
24    dfs(idx + 1);
25    path.pop_back();
26 }
27
28 int main() {
29     int n;
30     cin >> n;
31     nums.resize(n);
32     for (int i = 0; i < n; i++) {
33         cin >> nums[i];
34     }
35
36     dfs(0);
37     return 0;
38 }
```

- 每个元素有"选"和"不选"两种选择
- 递归到末尾时输出当前路径
- 这是搜索 / 回溯的基本模板

第九单元 · 课堂练习 9.1

练习1：递推求 $1! + 2! + 3! + \dots + n!$ 的和。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(0);
7
8     int n;
9     cin >> n;
10
11     long long fact = 1;        // 当前阶乘值
12     long long sum = 0;
13
14     for (int i = 1; i <= n; i++) {
15         fact *= i;              //  $i! = (i-1)! * i$ 
16         sum += fact;
17     }
18
19     cout << sum << endl;        //  $1!+2!+3!+4!+5! = 1+2+6+24+120 = 153$ 
20     return 0;
21 }
```

✎ 解析

- 关键技巧： `fact *= i` 递推计算阶乘，避免重复计算
- `fact` 就是 `i!`，直接累加即可
- 用 `long long` 防止溢出

练习2：约瑟夫环问题。

✓ 解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int josephus(int n, int k) {
5     // 方法：用 vector 模拟（直观但  $O(n^2)$ ）
6     vector<int> people;
7     for (int i = 1; i <= n; i++) {
8         people.push_back(i);
9     }
10
11     int idx = 0;
12     while (people.size() > 1) {
13         idx = (idx + k - 1) % people.size(); // 数到k的人
14         people.erase(people.begin() + idx); // 出局
15     }
16
17     return people[0];
18 }
19
20 int main() {
21     int n;
22     cin >> n;
23     cout << josephus(n, 3) << endl; // 数到3出局，n=10时输出4
24     return 0;
25 }
```

📝 解析

- 用 `vector` 模拟环，每次删除出局的人
- `idx = (idx + k - 1) % size` 计算要删除的位置
- 也可以用递归公式 $O(n)$ 解决

第九单元 · 课后作业 9

作业 1：杨辉三角前 n 行。

✓ 解答

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     vector<vector<int>> tri(n);
9
10    for (int i = 0; i < n; i++) {
11        tri[i].resize(i + 1, 1); // 每行 i+1 个元素, 初始为1
12
13        // 中间元素: tri[i][j] = tri[i-1][j-1] + tri[i-1][j]
14        for (int j = 1; j < i; j++) {
15            tri[i][j] = tri[i - 1][j - 1] + tri[i - 1][j];
16        }
17
18        // 输出
19        for (int j = 0; j <= i; j++) {
20            cout << tri[i][j] << " ";
21        }
22        cout << endl;
23    }
24
25    return 0;
26 }

```

解析

- 杨辉三角的递推公式: $C(n, k) = C(n-1, k-1) + C(n-1, k)$
- 每行首尾都是 1
- 可以用滚动数组优化空间到 $O(n)$

作业2: 铺砖问题 (1×1 和 1×2 铺满 $1 \times n$)。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long paveWays(int n) {
5     if (n <= 2) return n;
6
7     long long a = 1, b = 2; // dp[1]=1, dp[2]=2
8     for (int i = 3; i <= n; i++) {
9         long long temp = a + b;
10        a = b;
11        b = temp;
12    }
13    return b;
14 }
15
16 int main() {
17     int n;
18     cin >> n;
19     cout << paveWays(n) << endl;
20     return 0;
21 }
```

解析

- $dp[n] = dp[n-1] + dp[n-2]$ (放 1×1 或 1×2)
- 这本质上也是斐波那契数列
- 用滚动变量节省空间

作业3：辗转相除法 vs 更相减损术效率对比。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // 辗转相除法（欧几里得算法）
5 int gcd_euclid(int a, int b) {
6     while (b != 0) {
7         int temp = b;
8         b = a % b;
9         a = temp;
10    }
11    return a;
12 }
13
14 // 更相减损术
15 int gcd_subtract(int a, int b) {
16     while (a != b) {
17         if (a > b) a -= b;
18         else b -= a;
19     }
20     return a;
21 }
22
23 int main() {
24     int a, b;
25     cin >> a >> b;
26
27     // 测试辗转相除法
28     auto start1 = chrono::high_resolution_clock::now();
29     int r1 = gcd_euclid(a, b);
30     auto end1 = chrono::high_resolution_clock::now();
31     auto t1 = chrono::duration_cast<chrono::nanoseconds>(end1 - start
1).count();
32
33     // 测试更相减损术
34     auto start2 = chrono::high_resolution_clock::now();
35     int r2 = gcd_subtract(a, b);
36     auto end2 = chrono::high_resolution_clock::now();
37     auto t2 = chrono::duration_cast<chrono::nanoseconds>(end2 - start
2).count();
38
39     cout << "辗转相除法结果=" << r1 << ", 耗时=" << t1 << "ns" << endl;
40     cout << "更相减损术结果=" << r2 << ", 耗时=" << t2 << "ns" << endl;
```

```
41     cout << "两者结果一致：" << (r1 == r2 ? "是" : "否") << endl;
42
43     return 0;
44 }
```

解析

- 辗转相除法用取余运算，每次大幅缩小问题规模，时间复杂度 $O(\log \min(a,b))$
 - 更相减损术用减法，最坏情况如 $(N, 1)$ 需要 $O(\max(a,b))$ 次；相邻斐波那契数的减法次数约为 $O(\log \max(a,b))$
 - 结论：辗转相除法在绝大多数情况下更快
-

竞赛进阶 · 综合练习答案

综合题 1：网格不同路径数（只能向右或向下）。

解答

C/C++

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7
8     // dp[i][j] = 到达(i,j)的路径数
9     vector<vector<long long>> dp(n, vector<long long>(m, 0));
10
11     // 第一行和第一列只有1条路径
12     for (int i = 0; i < n; i++) dp[i][0] = 1;
13     for (int j = 0; j < m; j++) dp[0][j] = 1;
14
15     for (int i = 1; i < n; i++) {
16         for (int j = 1; j < m; j++) {
17             dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
18         }
19     }
20
21     cout << dp[n - 1][m - 1] << endl;
22     return 0;
23 }
```

解析

- 到达 (i,j) 只能从上方 (i-1,j) 或左方 (i,j-1) 来
- $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
- 这也是组合数学的 $C(n+m-2, n-1)$

综合题 2：N 皇后问题（输出所有方案数量）。

解答

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, ans = 0;
5 vector<int> cols;
6
7 bool isValid(int row, int col) {
8     for (int i = 0; i < row; i++) {
9         if (cols[i] == col) return false;
10        if (abs(cols[i] - col) == abs(i - row)) return false;
11    }
12    return true;
13 }
14
15 void dfs(int row) {
16     if (row == n) {
17         ans++;
18         return;
19     }
20     for (int col = 0; col < n; col++) {
21         if (isValid(row, col)) {
22             cols[row] = col;
23             dfs(row + 1);
24         }
25     }
26 }
27
28 int main() {
29     cin >> n;
30     cols.resize(n);
31     dfs(0);
32     cout << ans << endl;
33     return 0;
34 }
```

解析

- 每一行放置一个皇后，`cols[row]` 记录该行皇后的列号
- `isValid` 检查列冲突和两条对角线冲突，DFS 只向合法位置继续搜索
- 每当递归到 `row == n`，就找到一个完整方案，计数器加一；不提前返回，因此会统计所有方案

综合题 3： BFS 判断图是否连通。

 **解答**

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     vector<vector<int>> matrix(n, vector<int>(n));
9     for (int i = 0; i < n; i++) {
10         for (int j = 0; j < n; j++) {
11             cin >> matrix[i][j];
12         }
13     }
14
15     vector<bool> visited(n, false);
16     queue<int> q;
17
18     if (n == 0) {
19         cout << "图是连通的" << endl;
20         return 0;
21     }
22
23     q.push(0);
24     visited[0] = true;
25     int visited_count = 0;
26
27     while (!q.empty()) {
28         int node = q.front();
29         q.pop();
30         visited_count++;
31
32         for (int neighbor = 0; neighbor < n; neighbor++) {
33             if (matrix[node][neighbor] != 0 && !visited[neighbor]) {
34                 visited[neighbor] = true;
35                 q.push(neighbor);
36             }
37         }
38     }
39
40     if (visited_count == n) {
41         cout << "图是连通的" << endl;
42     } else {
```

```
43         cout << "图不是连通的，只访问了" << visited_count << "/" << n <<
    "个顶点" << endl;
44     }
45
46     return 0;
47 }
```

解析

- 输入的第二部分是 $n \times n$ 邻接矩阵，`matrix[u][v] != 0` 表示顶点 u 和 v 之间存在边
- 从顶点 0 开始 BFS，访问到的顶点数量等于 n 时，说明图连通
- 无向图的邻接矩阵通常关于主对角线对称；BFS 本身只依赖矩阵中的边关系

— 答案与解析文件结束 —

新哲文院算法社 2026-2027 学年